Dario Berti

Turin, Italy | +39 345 248 2954 | dario.berti100@gmail.com | linkedin.com/in/dario-berti-210843278 | github.com/DarioBerti

AI engineer currently building agentic LLM systems that can be trusted in regulated domains such as legal investigation. I focus on the problem rather than the tech: the right stack is whatever solves it reliably. Strong software engineering background in distributed systems and DevOps. Graduating March 2027, looking for an AI Engineer role.

Education

Politecnico di Torino — Turin, Italy
MSc in Computer Engineering (Software) — *Mar 2027 (expected)*
- Coursework: Large Language Models for Software Engineering, Cloud Programming & Operations (Kubernetes, DevOps), Software Engineering, Mobile Application Development, Model-Based Software Design (ISO 26262).

Alma Mater Studiorum – University of Bologna — Cesena, Italy
BSc in Computer Engineering — *Jul 2024*

Experience

AI Engineer, MSc Thesis *(confidential, code not public)* — Sept 2026 – Feb 2027
LINKS Foundation — *Turin, Italy*
- Architected and building an agentic LLM system modeling criminal investigations (Kotlin/Spring Boot, React/TS, Claude API). One of two developers, owning architecture and the LLM pipeline; working weekly with a Judicial Administrative Officer to turn criminal-procedure law into a faster, more reliable and machine-checkable process.
- To ensure legally sound and predictable agent behavior, I implemented a generator-validator architecture: an independent and deterministic validator checks the generated response against criminal-procedure rules, so the model never judges its own output and specific rejections autonomously guide the agent toward correction.
- Reduced hallucinations to near zero in expert-reviewed runs: immutable ground-truth story, facts cited only by ID, strict schema, closed error criteria and iteratively refined prompts. Benchmarked models on planted-error stories (detection, false positives, severity); the selected model's findings were reviewed and approved by the domain expert.
- To keep AI-generated code reliable, I built the workflow around the spec: every requirement is traced to a test, key decisions are recorded as ADRs, and CI gates every merge on lint, tests, E2E, security scans and $\geq$80% line coverage (currently 95%+).

Full-Stack Developer, Bachelor's Thesis: Crowdsensing Web App | GitHub — Mar 2024 – Jul 2024
Alma Mater Studiorum – University of Bologna — *Cesena, Italy*
- Cut the cost of mapping a city's street lighting by making the smartphone the only sensor needed, with no dedicated hardware: a crowdsensing web app (Vue.js 3, PHP REST API, MySQL) turns users' walks into brightness-coded routes on a Leaflet map. Sole developer; added SQL-injection-safe auth and badge gamification to retain contributors.

Projects

EuroTransit, a cloud-native ticket marketplace | GitHub: code, deployment | *Kotlin, Spring Boot, Kafka, Kubernetes* 2026
- Built with 3 teammates a train-ticket marketplace (6 Spring Boot microservices on Kubernetes, React/TypeScript SPA) designed to keep working correctly even when servers crash mid-checkout.
- To guarantee no booking is ever lost, duplicated or oversold, designed checkout as a Kafka saga with a transactional outbox (Debezium CDC) and idempotent consumers, and enforced inventory in the database itself: zero booking errors across 8 chaos experiments (pod kills, broker loss, Postgres failover), RPO=0.
- Made PayPal payments safe to retry with an idempotent authorize/capture gateway and a circuit breaker, so a timeout or retry never charges a customer twice.
- To catch checkout problems before customers notice, defined 4 measurable reliability targets (SLOs) with Prometheus burn-rate alerts that warn the team early, and verified them under k6 load.
- Automated delivery with GitOps (Argo CD, SealedSecrets) so CI never holds cluster credentials; an Argo Rollouts canary auto-aborted a deliberately broken release before users saw errors.

LetMeCook, an Android social recipe app | *Kotlin, Jetpack Compose, Firebase, CameraX* | GitHub — 2026
- Shipped with 3 teammates a native Android community app covering 12 feature areas (recipe publishing, reviews, grocery lists, search, notifications): Jetpack Compose UI of about 130 composables, Firebase Auth with Google Sign-In, and in-app camera capture for recipe photos.
- To keep every screen predictable and the feed live, structured the app as MVVM with 16 ViewModels exposing immutable per-screen UI state over a repository layer, and used Firestore snapshot listeners so the community feed and notifications update in real time without polling.

NASA Space Apps Challenge, hackathon | *48-hour team hackathon* — 2024
- Built in 48 hours an interactive educational tool explaining how NASA's DART mission deflects hazardous asteroids.

Technical Skills

Languages: Kotlin, Java, Python, TypeScript/JavaScript, SQL, PHP
AI & ML: Python, PyTorch, Hugging Face (model training and fine-tuning)
LLM & Agents: Claude API, tool calling, structured outputs, prompt engineering, LLM evaluation, generator-validator guardrails
Backend & Web: Spring Boot, REST APIs, Apache Kafka, PostgreSQL, MySQL, Debezium, React, Vue.js
Cloud & DevOps: Kubernetes, Docker, Helm, Argo CD (GitOps), GitHub Actions (CI/CD), Prometheus, Grafana, OpenTelemetry
Security & Reliability: OAuth2/OIDC (Keycloak), JWT, SealedSecrets, chaos testing (Chaos Mesh), load testing (k6)
Practices: Git, unit/integration testing (JUnit), UML, agile, AI-assisted development (Claude Code, agent context docs)
Spoken: Italian (native), English (C1 Advanced, IELTS), French (basic)